

Good Agentic Friends Do Not Just Give Verbal Advice: They Can Update Your Weights

Wenrui Bao
University of Central Florida

Huan Wang
Westlake University

Jian Wang
Snap Inc.

Zhangyang Wang
UT-Austin

Kai Wang
Tencent Hy

Yuzhang Shang*
University of Central Florida

[Project](#) [Code](#)

Abstract

Multi-agent LLM systems usually collaborate by exchanging natural-language messages. This interface is simple and interpretable, but it forces each sender’s intermediate computation to be serialized into tokens and then reprocessed by the receiver, thereby increasing the generated-token cost, prefill overhead, and KV-cache memory. We study an alternative communication interface: instead of appending a sender’s message to the receiver’s context, compile the sender’s hidden states into a transient, receiver-specific weight perturbation. We introduce **TFLOW** (Thought Flow), a weight-space communication framework for a known and fixed receiver architecture. For each query, frozen role-prompted sender agents process the input, and a learned parameter generator maps their internal activations into low-rank LoRA perturbations targeting the receiver’s modules. These perturbations are fused and applied only during the receiver’s generation, enabling instance-level adaptation without permanently changing the model or enlarging the receiver’s text context. With three Qwen3-4B agents, **TFLOW** improves over a standalone receiver by up to 8.5 accuracy points across five benchmarks while reducing processed tokens by up to 32.69%. Compared with a text-based three-agent baseline, it reduces total processed tokens by up to 83.27% and the wall-clock inference time by up to 4.6×, while maintaining competitive accuracy on four of five benchmarks. These results suggest that transient low-rank weight perturbations can serve as an executable communication medium for efficient multi-agent LLM collaboration.

1 Introduction

Multi-agent LLM systems are often built like small committees. One model proposes a strategy, another contributes domain knowledge, and a final model produces the answer. This design has been effective across reasoning, coding, and tool-use settings [1, 2, 3, 4, 5, 6, 7]. Yet the communication interface between agents has changed surprisingly little: most systems still require each agent to write a natural-language message that is appended to another agent’s context.

This text interface is attractive because it is universal and human-readable. **We yet argue it might be an awkward interface between neural networks.** A sender agent has already transformed the input into a rich internal state, but conventional multi-agent systems force this state to be decoded into tokens before the receiver can use it. The receiver must then encode those tokens again, paying additional prefill cost and storing a larger KV cache (Figure 1 (i)), whose memory and latency grow with context length [8, 9]. As the number of agents or communication rounds grows, this repeated write-read cycle can dominate the cost of collaboration. The issue is not only that natural-language

*Corresponding author

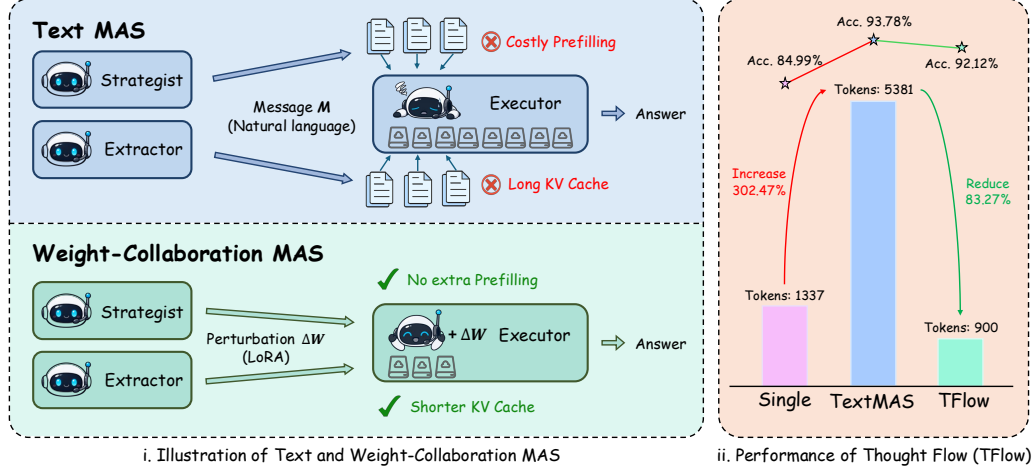


Figure 1: (i) Comparison between Text-based MAS and the proposed Weight-Collaboration MAS. In Text MAS, auxiliary agents transmit natural language messages to the Executor, incurring costly prefilling overhead and inflated KV cache. In contrast, our proposed paradigm compresses inter-agent communication into lightweight LoRA weight perturbations ΔW , which are directly merged into the parameters, thereby eliminating the extra prefilling and significantly reducing the KV cache footprint. (ii) Performance overview on GSM8K. TFlow achieves accuracy competitive with TextMAS while reducing token consumption by 76.7%, substantially surpassing the single-agent baseline in both accuracy and efficiency.

messages are long; it is that they make model-to-model communication pass through a channel designed for humans.

Recent work has begun to question this design by allowing agents to exchange continuous representations rather than text [10, 11, 12, 13, 14, 15]. These latent-space protocols reduce decoding overhead and can preserve information that would be difficult to express in words. However, they still require the receiver to consume sender information as activations, embeddings, or cache states. This creates a different compatibility problem: the transmitted object must be meaningful inside the receiver’s representation geometry, making such methods most natural when agents share architectures, hidden spaces, or carefully trained adapters.

We explore a complementary route. *Instead of sending information that the receiver must read, we send information that changes how the receiver computes.* In particular, we ask whether a sender’s internal state can be compiled into a temporary low-rank perturbation of the receiver’s weights [16, 17]. The receiver then solves the query under this perturbed computation, after which the perturbation is removed, and the frozen base model is restored. In this view, the communication object is not a sentence and not a hidden state injected into the receiver’s cache. It is a query-specific update in the receiver’s parameter space.

We instantiate this idea in **TFlow** (Thought Flow), a framework for weight-space communication among LLM agents. We focus on a practical fixed-receiver setting: the receiver architecture and target modules are known, all LLM backbones remain frozen, and only a receiver-specific parameter generator [18, 19] is trained. Given a query, each sender agent processes the prompted input once and exposes its hidden states as a conditioning signal. The parameter generator maps this signal into layer- and module-specific LoRA factors for the receiver. Perturbations from multiple senders are fused and injected by transiently patching the forward pass of the receiver’s target linear layers. The receiver then generates its answer without seeing any sender-written message in its text context (Figure 1 (ii)).

This design separates TFlow from both text-based and latent-space collaboration. Compared with text-based MAS, TFlow avoids expanding the receiver’s input with auxiliary messages, thereby reducing generated-token usage, prefill overhead, and KV-cache growth. Compared with latent-state transfer, TFlow does not ask the receiver to directly interpret a sender’s activations. Sender states are instead translated by a learned generator into perturbations that are compatible with the receiver’s own weight tensors by construction. The result is an executable communication channel: sender information affects the receiver by modifying the computation path used for the current query.

We evaluate TFLOW in a three-agent setting with a frozen Qwen3-4B backbone, where sender agents provide different perspectives through hidden states and the receiver produces the final answer. Across five benchmarks, TFLOW improves over the standalone receiver by up to 8.5 accuracy points while reducing total processed tokens by up to 32.69%. Relative to a text-based three-agent baseline, it cuts processed tokens by up to 83.27% and wall-clock inference time by up to 4.6x, with competitive accuracy on four of five benchmarks; the remaining gap on HumanEval+ suggests that additional generation budget matters more than inter-agent communication for code synthesis.

Our contributions are:

- Motivated by the high communication cost of text-based multi-agent systems, we propose *weight-space communication*, a new communication paradigm for multi-agent LLM collaboration, in which sender information is transmitted as transient low-rank perturbations to a frozen receiver model rather than as text appended to the receiver’s context.
- Under this paradigm, we introduce **TFLOW**, a receiver-specific framework that transforms sender hidden states into query-specific LoRA factors, integrates multiple sender contributions, and injects the resulting perturbation only during the receiver’s generation.
- We demonstrate that TFLOW achieves substantial efficiency gains across five reasoning, knowledge, and coding benchmarks, dramatically reducing generated-token and latency costs relative to text-based multi-agent collaboration with only negligible performance degradation, while consistently outperforming a standalone receiver.

2 Related Work

2.1 LLM-Based Multi-Agent Systems

Multi-Agent Collaboration Paradigms. Large language models have catalyzed a wave of multi-agent systems. Early explorations such as CAMEL [1] and Generative Agents [20] demonstrate the power of role-playing dialogue, enabling agents to cooperate through structured natural-language conversations. Subsequent work applies multi-agent collaboration to complex engineering workflows: MetaGPT [2] and ChatDev [21] mirror real software companies by assigning specialized roles and routing structured artifacts between them, while AutoGen [3] and AutoAgents [22] provide general-purpose frameworks for orchestrating and automatically generating task-specific agents. Another work leverages multi-agent debate to improve reasoning quality [4, 5]. ReConcile [23] organizes round-table conferences among diverse LLM families with confidence-weighted voting. More recently, Mixture-of-Agents [24] and [25] show that iteratively aggregating or even naively scaling diverse model outputs can surpass the strongest individual model. Despite their architectural diversity, all of the above systems share a common reliance on natural language as the primary medium for inter-agent communication.

Inter-Agent Communication Mechanisms. The communication mechanism is central to the effectiveness of any multi-agent system. The prevailing paradigm transmits information as natural-language messages concatenated into each receiver’s context window [1, 2, 3, 21]. While natural language provides a universal and human-interpretable interface, serializing an agent’s intermediate computation into tokens may lose information that is difficult to verbalize and can introduce substantial communication overhead [26, 13, 4, 27]. Recent work has therefore explored non-textual communication channels: CIPHER [26] lets agents communicate through probability-weighted output embeddings; Interlat [12] transmits hidden states as a compressed representation of an agent’s internal computation; and [13, 28, 14] further formalize and extend latent-space communication. These approaches show that moving beyond natural-language messages can preserve richer intermediate signals and reduce token-level overhead, primarily by communicating in activation or embedding space. In this work, we explore a complementary paradigm: *weight-space communication*, where sender-side computation is converted into targeted, instance-specific perturbations of the receiver’s model parameters, which guide the receiver’s generation.

2.2 Knowledge Representation and Operations in Weight Space

Weight-Space Structure and Static Merging. A growing body of evidence shows that neural network weight spaces are structured carriers of transferable knowledge [29, 30]. The *task vector*

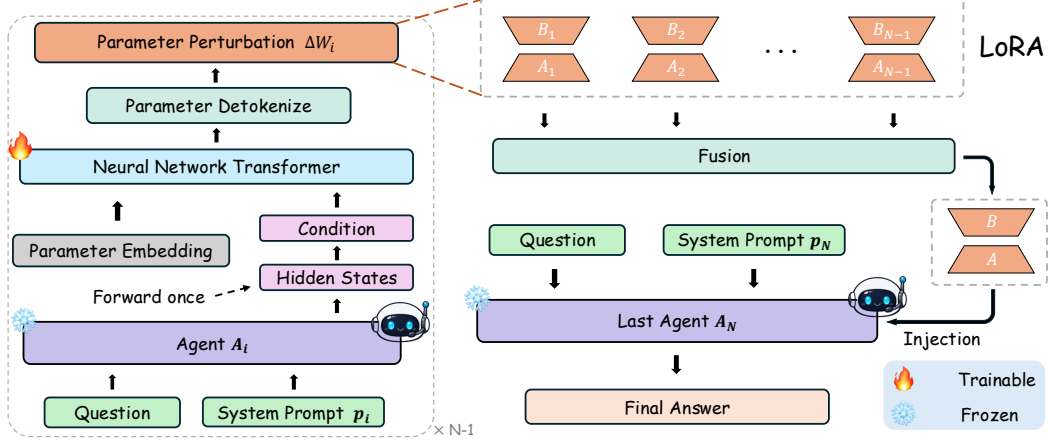


Figure 2: **Overview of TFLOW.** TFLOW realizes multi-agent collaboration through dynamic, instance-specific parameter generation. Given an input question, each sender agent A_i receives the question together with its system prompt p_i and performs a single frozen forward pass to produce hidden-state representations that summarize its role-specific reasoning signal. These hidden states serve as conditions for a trainable **parameter generator**, which maps them into parameter embeddings, applies a neural transformation, and detokenizes the generated representation into a low-rank perturbation ΔW_i , parameterized by **LoRA factors** (A_i, B_i). The per-sender LoRA factors from all $N - 1$ agents are subsequently combined by a fusion module to form a unified update ΔW . This perturbation is transiently injected into the frozen receiver agent A_N , which is conditioned on the original question and its own system prompt p_N to generate the final answer. The injected parameters are created on demand for each input and are discarded after generation, leaving the receiver parameters unchanged. During training, all agents remain **frozen** and only the parameter generator is **trainable**.

framework [31] demonstrates that the element-wise difference between a fine-tuned model and its pretrained initialization encodes the learned task skill. Building on this insight, a family of model merging methods, including Model Soups [32], Fisher merging [33], TIES-Merging [34], and DARE [35], seeks to combine the knowledge of multiple fine-tuned models directly in weight space without additional training. Despite these advances, all of the above approaches operate in a *static* manner, in which the merging coefficients or perturbation directions are fixed once determined and do not adapt to individual inputs at inference time.

Low-Rank Adaptation and Dynamic Weight Generation. LoRAHub [36] consists of multiple LoRA modules with learned scalar weights for generalizing the cross-task, while LoRAs composition [37] and LoRA-Flow [38] further enable selection of dynamic adapters at the instance-level and layer-wise, respectively. Going one step further, recent approaches use hypernetworks [19] to directly *generate* adapter parameters conditioned on input context—from task embeddings [18], to few-shot demonstrations [39], to full documents [40], which eliminates the need for pre-trained adapter libraries [41, 42, 43, 44, 45].

Our framework, TFLOW, extends this progression. Rather than conditioning on static documents or task embeddings, TFLOW uses a parameter generator to translate the *dynamic reasoning states* of multiple sender agents into low-rank perturbations of a receiver model’s weights.

3 TFLOW: Thought Flow

Figure 2 illustrates the overall pipeline of TFLOW. This section formalizes the setup (Section 3.1), describes how sender hidden states are aggregated into query-dependent conditioning signals (Section 3.2), introduces the parameter generator that emits LoRA factors (Section 3.3), and details the transient injection into the receiver (Section 3.4).

3.1 Problem Setup of Weight-Collaboration MAS

We consider a collaboration of N agents $\{A_1, \dots, A_N\}$ that share a single frozen backbone f_θ but are individuated by role-specific system prompts $\{p_1, \dots, p_N\}$. Given a query q , agents $A_1, \dots, A_{N-1}$ act as *senders* and A_N as the *receiver* [1, 3, 2]. The senders contribute only their hidden-state trajectories, while the receiver is the one who actually decodes the answer. Rather than exchanging tokens, the senders induce, for this single query, a transient parameter displacement applied to a designated set of linear modules inside the receiver:

$$\hat{\theta} = \theta + \Delta \mathbf{W}(q, \{p_i\}_{i=1}^{N-1}; \psi). \quad (1)$$

Here ψ denotes the parameters of a learned generator and is the *only* set of weights updated during training. The backbone tensors themselves are never overwritten: the perturbation is implemented as a temporary forward patch and removed immediately after the receiver call, so different queries always observe the same original backbone.

Two design constraints follow directly and will shape every component below. First, because we rely on the existence of beneficial neighbors rather than on aggressive fine-tuning, $\Delta \mathbf{W}$ must remain a *low-rank, small-norm* displacement. Second, because useful perturbation directions are *instance-specific*, the generator must be conditioned on representations that depend on q itself, not merely on the static role descriptors p_i . We therefore condition on the senders' hidden states over the prompted input, which simultaneously encodes the role p_i and the content of the query.

Notation. In the remainder of this section, we restrict weight-space perturbations to L selected decoder layers, with M linear modules perturbed in each selected layer. For the m -th perturbed module in the l -th selected layer, we denote its frozen weight matrix by $\mathbf{W}_m^{(l)}$.

3.2 Sender Conditioning

Each sender A_i encodes the prompted input (p_i, q) using the frozen backbone in a single forward pass. We retain the intermediate activations and denote the resulting hidden states at layer l by $\mathbf{H}_i^{(l)} \in \mathbb{R}^{T_i \times d}$, for $l = 0, 1, \dots, L_{\text{total}}$.

Learnable layer aggregation. Instead of relying on a single-layer representation, we aggregate each sender's hidden states across layers using temperature-scaled softmax weights over learnable layer-wise scalars. The resulting conditioning representation $\mathbf{C}_i \in \mathbb{R}^{T_i \times d}$ enables the generator to exploit features at different levels of abstraction while preserving a fixed conditioning dimensionality.

3.3 Parameter Generator

The parameter generator $\mathcal{H}_\psi$ maps each sender conditioning representation $\mathbf{C}_i$ to a complete set of LoRA factors spanning all $L \times M$ targeted modules in the receiver. The generator comprises three stages: conditioning-driven initialization, a multi-axis Transformer trunk, and detokenization into LoRA matrices.

Stage 1: Initialization. We construct a token grid of shape $(L, H \times W, d_{\text{pg}})$, where $H = H_A + H_B$ divides tokens into **A**- and **B**-generating groups, and W spans the target modules and rank slots. Learnable grid queries $\mathbf{Q}_{\text{grid}} \in \mathbb{R}^{L \times HW \times d_{\text{pg}}}$ are initialized by cross-attending to the projected conditioning representation $\text{Proj}_{\text{in}}(\mathbf{C}_i)$. We then add layer-index and module-rank positional embeddings to encode each token's structural location within the receiver.

Stage 2: Neural Network Transformer. Our generator architecture follows the structured parameter tokenization paradigm introduced by HY-WU [46]. The initialized grid is refined through N_{pg} stacked blocks, each applying three residual attention operations along complementary axes, followed by a feed-forward network. Cross-layer self-attention (SA_L), batched over the HW axis, captures dependencies across receiver layers; intra-layer self-attention (SA_{HW}), batched over the L axis, propagates information across modules and rank slots within each layer; and conditioning cross-attention (CA) re-anchors the generation process to the sender representation. All attention operations employ RoPE [47] along their active sequence axis.

Stage 3: Detokenization. After N_{pg} blocks, the refined grid is partitioned along the H dimension into A- and B-tokens. Two linear heads map these tokens to parameter slices, which are rearranged into per-sender LoRA factors $\mathbf{A}_{i,m}^{(l)} \in \mathbb{R}^{r \times d_{\text{in}}}$ and $\mathbf{B}_{i,m}^{(l)} \in \mathbb{R}^{d_{\text{out}} \times r}$. The generator is shared across all senders; sender-specific factors are induced solely by the conditioning representation $\mathbf{C}_i$.

3.4 Transient Injection

The generated factors are injected into each targeted module of the receiver as a low-rank weight offset. When multiple senders are active, their generated LoRA updates are fused by a lightweight scalar gate before being applied to the receiver:

$$\Delta \mathbf{W}_m^{(l)} = \frac{\alpha}{r} \sum_{i=1}^{N-1} \text{softmax}_i(g_{\mathbf{w}}(\mathbf{C}_i)) \mathbf{B}_{i,m}^{(l)} \mathbf{A}_{i,m}^{(l)}. \quad (2)$$

The resulting $\Delta \mathbf{W}_m^{(l)}$ is injected only into the designated receiver module m at layer l . During the receiver’s forward pass, the frozen linear map is evaluated with this transient additive update. After generation, the patch is discarded, ensuring that each input induces its own temporary parameterization and that all subsequent inputs start from the same frozen backbone.

3.5 Training Objective

Diversity regulariser. To prevent the generator from collapsing toward a query-independent LoRA, we penalise the squared cosine similarity $\mathcal{L}_{\text{div}}(\Theta; x_q) = \cos^2(v(x_q), v_{\text{prev}}^{(\sigma(x_q))})$ between the flattened LoRA vector $v(x_q)$ of the current sample and the cached vector $v_{\text{prev}}^{(\sigma)}$ from the previous step. This loss is minimised at orthogonality, equally penalises parallel and anti-parallel collapse, and incurs no additional forward cost since the cache is updated via stop-gradient after each backward pass.

End-to-end optimization. The task loss $\mathcal{L}_{\text{task}}$ is the standard masked next-token cross-entropy computed only over completion tokens, with the receiver conditioned on both the chat-templated prompt and the injected ΔW . The full training objective combines it with the diversity regulariser:

$$\mathcal{L}(\Theta) = \mathbb{E}_{(x_q, t, \sigma) \sim \mathcal{D}} [\mathcal{L}_{\text{task}}(\Theta; x_q, t) + \lambda_{\text{div}} \mathcal{L}_{\text{div}}(\Theta; x_q)]. \quad (3)$$

The trainable parameter set Θ encompasses all components of the parameter generator $\mathcal{H}_{\psi}$, the conditioning projection layers, and the gating head; the receiver and sender backbones share the frozen parameters θ and remain entirely frozen throughout training. We do not add auxiliary weight-norm or inter-sender cosine penalties in our default objective; empirically, the cache-based diversity regularizer is sufficient to encourage instance-level specialization.

4 Experiments

4.1 Experimental Setups

Datasets and evaluation. We evaluate on five benchmarks spanning mathematical reasoning, code generation, and multi-disciplinary knowledge: GSM8K [48], MATH [49], MMLU [50, 51], HumanEval+ [52, 53], and MBPP+ [52, 54]. All benchmarks are evaluated under the zero-shot, chain-of-thought setting with no in-context exemplars. At inference time, all methods use identical decoding hyperparameters, which is temperature 0.6, top- p 0.95.

Backbone and baselines. All methods share a single frozen Qwen3-4B [55] backbone, whose parameters are never updated during parameter generator training or inference. We compare against three baselines: (i) **Single-Agent**, which directly prompts Qwen3-4B with a task-level system prompt and represents the backbone’s intrinsic capability; (ii) **TextMAS**, a three-agent text-level collaboration in which Agents A and B produce strategy and domain-knowledge passages that are concatenated into Agent C’s context, representing conventional multi-agent cooperation.

Table 1: **Main results across five benchmarks.** For each task, we report accuracy (**Acc.** $\uparrow$, %), average total processed tokens (**Token** $\downarrow$), and end-to-end wall-clock inference time (**Speed**, in seconds). Single, TextMAS, and TFLOW are shown side by side. The **Improvement** columns compare TFLOW with each baseline: $\uparrow/\downarrow$ denote accuracy increases/decreases in absolute points, $\downarrow$ denotes relative token reduction, and $\times$ denotes the speed ratio. **Green** indicates favorable changes, while **red** indicates unfavorable changes.

Tasks	Metrics	Baselines		TFLOW	Improvement	
		Single	TextMAS		Δ Single	Δ TextMAS
General Task						
MMLU-Redux	Acc. $\uparrow$	58.99	71.50	66.97	$\uparrow$ 7.98	$\downarrow$ 4.53
	Token $\downarrow$	1079	4825	998	$\downarrow$ 7.51 %	$\downarrow$ 79.32 %
	Speed (s)	8226	36450	9784	$\times$ 0.84	$\times$ 3.73
Math & STEM Task						
GSM8K	Acc. $\uparrow$	84.99	93.78	92.12	$\uparrow$ 7.13	$\downarrow$ 1.66
	Token $\downarrow$	1337	5381	900	$\downarrow$ 32.69 %	$\downarrow$ 83.27 %
	Speed (s)	6230	27256	5953	$\times$ 1.05	$\times$ 4.58
MATH	Acc. $\uparrow$	16.18	26.47	23.16	$\uparrow$ 7.98	$\downarrow$ 3.31
	Token $\downarrow$	2782	8188	2242	$\downarrow$ 19.41 %	$\downarrow$ 72.62 %
	Speed (s)	1984	5213	2258	$\times$ 0.88	$\times$ 2.31
Coding Task						
MBPP+	Acc. $\uparrow$	59.79	68.52	67.20	$\uparrow$ 7.41	$\downarrow$ 1.32
	Token $\downarrow$	1533	5500	1301	$\downarrow$ 15.13 %	$\downarrow$ 76.35 %
	Speed (s)	1998	5796	2395	$\times$ 0.83	$\times$ 2.42
Humaneval+	Acc. $\uparrow$	56.71	75.00	65.24	$\uparrow$ 8.53	$\downarrow$ 9.76
	Token $\downarrow$	1756	5879	1662	$\downarrow$ 5.35 %	$\downarrow$ 71.73 %
	Speed (s)	872	2512	1065	$\times$ 0.82	$\times$ 2.36

Parameter generator configuration. The system deploys $N = 3$ agents sharing the same backbone instance: Agent A analyzes reasoning types and plans chain-of-thought structure, Agent B retrieves domain knowledge and implicit constraints, and Agent C solves the problem after the generated LoRA perturbations are injected. Specific system prompts are designed for each role. The parameter generator is a lightweight Transformer ($d_{\text{pg}} = 1024$, head dimension 128, $N_{\text{pg}} = 2$ blocks, token dimension 256, dimension-accumulation factor $c = 2$) that produces LoRA factors of rank $r = 4$ with scaling $\alpha = 8$ ($s = \alpha/r = 2$).

Training details. The training data consist of OpenThoughts [56], Sky-T1 [57], and KodCode [58]. We train for one epoch on 32,000 samples using a single NVIDIA RTX PRO 6000, which takes approximately 8 hours.

4.2 Main Results

Table 1 reports accuracy, average total processed token usage, and wall-clock inference time for TFLOW, the Single-Agent baseline, and TextMAS across all five benchmarks.

Accuracy improvement over Single-Agent. TFLOW consistently outperforms the single-agent baseline across all task categories, with accuracy gains of +7.13 to +8.53 points. Notably, these gains come with *fewer* total processed tokens on every benchmark, indicating that weight-space perturbations guide the receiver toward more concise reasoning paths rather than verbose exploration. TFLOW incurs higher end-to-end latency on four of the five benchmarks, because applying instance-specific LoRA perturbations to the backbone prevents efficient batched generation.

Efficiency advantage over TextMAS. As shown in Table 1, total processed-token consumption is cut by 71–83% across all five benchmarks, translating to 2.3–4.6 $\times$ wall-clock speed-ups. These gains

stem from weight-space injection, entirely bypassing the lengthy prefilling and KV-cache overhead inherent in text-based context concatenation. Compared with TextMAS, the accuracy gap remains modest—within 1.3–4.5 points on four of the five tasks. The sole outlier is HumanEval+ ($\Delta = 9.76$). We attribute this to TextMAS’s concatenated context serving a dual role: beyond transferring inter-agent knowledge, it implicitly extends the generation budget, enabling longer outputs that facilitate structural elaboration and self-debugging. TFLOW successfully transfers collaborative knowledge through the weight space but does not alter the output distribution’s length characteristics, a distinction that surfaces most clearly in code generation.

4.3 Instance-level TFLOW Analysis

A core property of TFLOW is that the LoRA consumed by the receiver is generated *per input*, rather than learned once and reused. We verify this by examining two points of the pipeline: the conditioning vector $\mathbf{c}$ that drives the parameter generator (section 4.3.1), and the resulting LoRA tensors for the receiver (section 4.3.2).

4.3.1 Hidden-State Evidence

We extract per-layer hidden states for 20 instances per dataset and report mean pairwise cosine similarity in three regimes: within-GSM8K, within-MBPP+, and cross-task (GSM8K $\times$ MBPP+).

Learned layer weights. As shown in Figure 3, the learned weights $\{w_\ell\}$ do *not* simply track the lowest pairwise-similarity layers. More than 80% of the mass concentrates on a small set of layers (L05, L29, L30), each with a distinct similarity profile. This selection emerges purely from end-to-end training, suggesting that these layers are chosen for their conditioning utility rather than surface-level diversity.

Pairwise similarity across regimes. Figure 3 further shows that the aggregated conditioning vector preserves the expected ordering of semantic similarity: within-GSM8K pairs (0.94) are most similar, followed by within-MBPP+ pairs (0.83), with cross-task pairs (0.73) being the least similar. This confirms that the conditioning representation encodes instance-level variation in a structured manner.

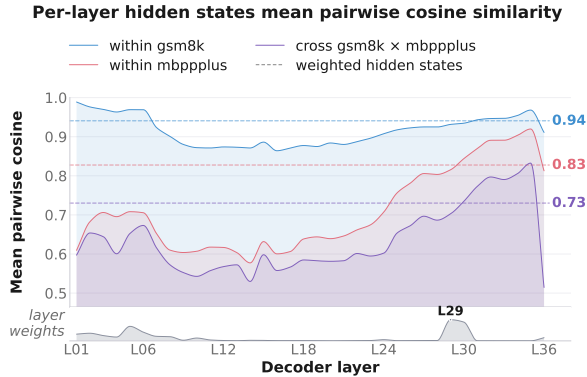


Figure 3: Per-layer mean **pairwise cosine similarity** of the question’s last-token hidden state across decoder layers. Solid curves report within-task similarity and cross-task similarity, respectively. Dashed lines indicate the corresponding similarity computed on the aggregated conditioning vector $\mathbf{c}$ that the parameter generator actually consumes. The bottom panel shows the learned layer weights $\{w_\ell\}$, which peak sharply at layer 29.

4.3.2 LoRA Tensor Evidence

For each instance, we materialise the effective delta-weights $\Delta W_k = B_k A_k$ across all $K=252$ adapted projections (attention q, k, v, o and MLP up, gate, down on every decoder layer) and concatenate them into a single gauge-invariant fingerprint θ . We then compute the mean pairwise cosine similarity of θ across $N=20$ instances on five benchmarks spanning three domains. Table 2 reports the resulting similarity matrix.

Distinct LoRAs per instance. The within-dataset similarity ranges from 0.264 to 0.933, reflecting the heterogeneity of the underlying instances: Python-coding tasks vary widely in specification, whereas olympiad-style MINERVA-MATH problems share a highly templated structure. This spread indicates that TFLOW naturally allocates more adapter variation to datasets with diverse instances and less to near-templated ones. This is exactly the behaviour one would expect of an architecture that genuinely conditions on the input rather than collapsing to a fixed per-task adapter.

Table 2: Mean pairwise cosine similarity of the gauge-invariant LoRA fingerprint θ ($N=20$ instances per dataset). Diagonal: within-dataset; off-diagonal: cross-dataset.

	GSM8K	MINERVA-MATH	MMLU	MBPP+	HUMANEVAL+
GSM8K	0.519	0.267	0.148	0.133	0.094
MINERVA-MATH	—	0.933	0.215	0.187	0.220
MMLU	—	—	0.424	0.063	0.099
MBPP+	—	—	—	0.264	0.252
HUMANEVAL+	—	—	—	—	0.315

Domain-aware geometry. The off-diagonal entries exhibit clear block structure: within-domain pairs consistently show higher similarity than cross-domain pairs. The math block and the code block are the two highest cross-dataset entries, while the math $\times$ code crossings are the lowest. Notably, the within-dataset similarity of MBPP+ nearly coincides with its cross-similarity to HUMANEVAL+, which is expected given that both are Python function-level benchmarks with overlapping prompt formats. In all cases the diagonal dominates its row, confirming that θ encodes task semantics faithfully.

4.4 Ablation Study: Instance-Specificity of Weight Perturbations

A key design choice in TFLOW is to generate *instance-specific* perturbations: the LoRA factors injected into the receiver are conditioned on both the sender’s reasoning and the particular input. To validate that this instance-level conditioning is essential to the observed gains, we conduct two ablations that progressively degrade specificity while holding all other variables constant.

Static LoRA. To isolate instance-conditioned parameter generation from standard parameter-efficient adaptation, we replace the parameter generator of TFLOW with a conventional LoRA adapter shared across all inputs, while keeping the backbone, adapted modules, LoRA rank, and training data unchanged.

As shown in Figure 4, Static-LoRA consistently improves over the Single-Agent baseline on all benchmarks. However, TFLOW achieves substantially stronger performance, outperforming Static-LoRA by 4.29 points on average, with especially clear gains on more challenging reasoning and code-oriented benchmarks such as MBPP+ and HumanEval+. This shows that TFLOW’s advantage goes beyond added trainable capacity and stems from input-dependent modulation of the receiver.

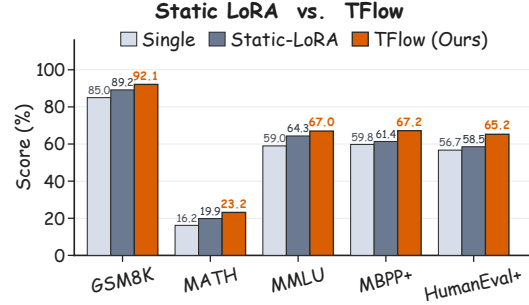


Figure 4: **Static LoRA vs. TFLOW** performance.

Mismatched Perturbation Injection. We further investigate whether TFLOW perturbations encode instance-level information by fixing the receiver input and replacing its matched perturbation with LoRA factors from random sources, other tasks, same-task samples, or the matched sample.

Figure 5 shows that random LoRA perturbations bring only marginal gains, while cross-task perturbations still improve over the baseline, indicating that TFLOW captures transferable collaborative signals. However, same-task perturbations perform better, and the matched-sample perturbation achieves the highest accuracy, suggesting that TFLOW encodes not only task-level knowledge but also fine-grained instance-specific cues. This confirms that the generated perturbations are meaningfully tied to the target input, rather than serving as generic task-level or adapter-like updates.

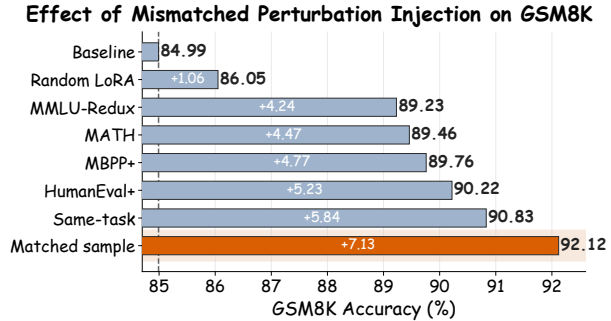


Figure 5: **Mismatched perturbation injection** on GSM8K.

5 Conclusion

We presented TFLOW, a weight-space collaboration paradigm for multi-agent LLM systems. Rather than routing inter-agent knowledge through natural language, TFLOW maps sender agents' hidden states into query-specific low-rank perturbations of a frozen receiver model. These perturbations are fused and transiently injected at inference time, enabling instance-level adaptation without increasing context length or requiring latent-space compatibility. Across five benchmarks, TFLOW consistently outperforms the single-agent baseline and achieves competitive accuracy with text-based multi-agent systems while using substantially fewer tokens and lower latency. Our findings indicate that model weights can serve as an effective communication medium among agents, suggesting a promising direction for efficient and scalable multi-agent LLM collaboration.

References

- [1] Li, G., H. Hammoud, H. Itani, et al. Camel: Communicative agents for "mind" exploration of large language model society. In *NeurIPS*. 2023.
- [2] Hong, S., M. Zhuge, J. Chen, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *ICLR*. 2023.
- [3] Wu, Q., G. Bansal, J. Zhang, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversations. In *COLM*. 2024.
- [4] Du, Y., S. Li, A. Torralba, et al. Improving factuality and reasoning in language models through multiagent debate. In *ICML*. 2024.
- [5] Liang, T., Z. He, W. Jiao, et al. Encouraging divergent thinking in large language models through multi-agent debate. In *EMNLP*. 2024.
- [6] Guo, T., X. Chen, Y. Wang, et al. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*, 2024.
- [7] Xi, Z., W. Chen, X. Guo, et al. The rise and potential of large language model based agents: A survey. *Science China Information Sciences*, 68(2):121101, 2025.
- [8] Pope, R., S. Douglas, A. Chowdhery, et al. Efficiently scaling transformer inference. In *MLSys*. 2023.
- [9] Kwon, W., Z. Li, S. Zhuang, et al. Efficient memory management for large language model serving with PagedAttention. In *SOSP*. 2023.
- [10] Li, X. L., P. Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *ACL-IJCNLP*. 2021.
- [11] Lester, B., R. Al-Rfou, N. Constant. The power of scale for parameter-efficient prompt tuning. In *EMNLP*. 2021.
- [12] Du, Z., R. Wang, H. Bai, et al. Enabling agents to communicate entirely in latent space. *arXiv preprint arXiv:2511.09149*, 2025.
- [13] Zheng, Y., Z. Zhao, Z. Li, et al. Thought communication in multiagent collaboration. In *NeurIPS*. 2025.
- [14] Zou, J., X. Yang, R. Qiu, et al. Latent collaboration in multi-agent systems. *arXiv preprint arXiv:2511.20639*, 2025.
- [15] Jin, H., K. Peng, Y. Yu, et al. Agent primitives: Reusable latent building blocks for multi-agent systems. *arXiv preprint arXiv:2602.03695*, 2026.
- [16] Houlisby, N., A. Giurgiu, S. Jastrzebski, et al. Parameter-efficient transfer learning for NLP. In *ICML*. 2019.
- [17] Hu, E. J., Y. Shen, P. Wallis, et al. LoRA: Low-rank adaptation of large language models. In *ICLR*. 2022.

- [18] Mahabadi, R. K., S. Ruder, M. Dehghani, et al. Parameter-efficient multi-task fine-tuning for transformers via shared hypernetworks. In *ACL-IJCNLP*. 2021.
- [19] Ha, D., A. Dai, Q. V. Le. Hypernetworks. *arXiv preprint arXiv:1609.09106*, 2016.
- [20] Park, J. S., J. O’Brien, C. J. Cai, et al. Generative agents: Interactive simulacra of human behavior. In *UIST*. 2023.
- [21] Qian, C., W. Liu, H. Liu, et al. ChatDev: Communicative agents for software development. In *ACL*. 2024.
- [22] Chen, G., S. Dong, Y. Shu, et al. Autoagents: A framework for automatic agent generation. *arXiv preprint arXiv:2309.17288*, 2023.
- [23] Chen, C. Y., S. Saha, M. Bansal. Reconcile: Round-table conference improves reasoning via consensus among diverse LLMs, 2024.
- [24] Wang, J., J. Wang, B. Athiwaratkun, et al. Mixture-of-agents enhances large language model capabilities. In *ICLR*. 2025.
- [25] Li, J., Q. Zhang, Y. Yu, et al. More agents is all you need. *TMLR*, 2024.
- [26] Pham, C., B. Liu, Y. Yang, et al. Let models speak ciphers: Multiagent debate through embeddings. In *ICLR*. 2024.
- [27] Smit, A. P., N. Grinsztajn, P. Duckworth, et al. Should we be going MAD? a look at multi-agent debate strategies for LLMs. In *ICML*. 2024.
- [28] Tang, Y., W. Su, Y. Zhou, et al. Augmenting multi-agent communication with state delta trajectory. In *EMNLP*. 2025.
- [29] Yang, E., L. Shen, G. Guo, et al. Model merging in LLMs, MLLMs, and beyond: Methods, theories, applications, and opportunities. *ACM Computing Surveys*, 58(8):1–41, 2026.
- [30] Li, W., Y. Peng, M. Zhang, et al. Deep model fusion: A survey. *arXiv preprint arXiv:2309.15698*, 2023.
- [31] Ilharco, G., M. T. Ribeiro, M. Wortsman, et al. Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.
- [32] Wortsman, M., G. Ilharco, S. Y. Gadre, et al. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *ICML*. 2022.
- [33] Matena, M. S., C. A. Raffel. Merging models with fisher-weighted averaging. In *NeurIPS*. 2022.
- [34] Yadav, P., D. Tam, L. Choshen, et al. Ties-merging: Resolving interference when merging models. *NeurIPS*, 2023.
- [35] Yu, L., B. Yu, H. Yu, et al. Language models are super mario: Absorbing abilities from homologous models as a free lunch. In *ICML*. 2024.
- [36] Huang, C., Q. Liu, B. Y. Lin, et al. LoraHub: Efficient cross-task generalization via dynamic LoRA composition. In *COLM*. 2024.
- [37] Wang, Z., S. He, K. Liu, et al. Instance-level dynamic LoRAs composition for cross-task generalization. In *Findings of EMNLP*. 2024.
- [38] Wang, H., B. Ping, S. Wang, et al. LoRA-Flow: Dynamic LoRA fusion for large language models in generative tasks. In *ACL*. 2024.
- [39] Phang, J., Y. Mao, P. He, et al. Hypertuning: Toward adapting large language models without back-propagation. In *ICML*. 2023.
- [40] Charakorn, R., E. Cetin, S. Uesaka, et al. Doc-to-lora: Learning to instantly internalize contexts. *arXiv preprint arXiv:2602.15902*, 2026.

- [41] Liang, Z., D. Tang, Y. Zhou, et al. Drag-and-drop LLMs: Zero-shot prompt-to-weights. In *NeurIPS*. 2025.
- [42] Jin, X., K. Wang, D. Tang, et al. Conditional LoRA parameter generation. *arXiv preprint arXiv:2408.01415*, 2024.
- [43] Wang, K., D. Tang, B. Zeng, et al. Neural network diffusion. *arXiv preprint arXiv:2402.13144*, 2024.
- [44] Wang, K., D. Tang, W. Zhao, et al. Scaling up parameter generation: A recurrent diffusion approach. In *NeurIPS*. 2025.
- [45] Han, X., F. Neri, Z. Jiang, et al. W2t: Lora weights already know what they can do. *arXiv preprint arXiv:2603.15990*, 2026.
- [46] Team, T. H., et al. Hy-wu (part i): An extensible functional neural memory framework and an instantiation in text-guided image editing. *arXiv preprint arXiv:2603.07236*, 2026.
- [47] Su, J., M. Ahmed, Y. Lu, et al. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- [48] Cobbe, K., V. Kosaraju, M. Bavarian, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [49] Hendrycks, D., C. Burns, S. Kadavath, et al. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- [50] Gema, A. P., J. O. J. Leang, G. Hong, et al. Are we done with MMLU? In *NAACL-HLT*. 2025.
- [51] Hendrycks, D., C. Burns, S. Basart, et al. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.
- [52] Liu, J., C. S. Xia, Y. Wang, et al. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *NeurIPS*. 2023.
- [53] Chen, M., J. Tworek, H. Jun, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [54] Austin, J., A. Odena, M. Nye, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- [55] Yang, A., A. Li, B. Yang, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [56] Guha, E., R. Marten, S. Keh, et al. Openthoughts: Data recipes for reasoning models. *arXiv preprint arXiv:2506.04178*, 2025.
- [57] Team, N. Sky-t1: Train your own o1 preview model within \$450. <https://novasky-ai.github.io/posts/sky-t1>, 2025. Accessed: 2025-01-09.
- [58] Xu, Z., Y. Liu, Y. Yin, et al. KodCode: A diverse, challenging, and verifiable synthetic dataset for coding. In *Findings of ACL*. 2025.

A Limitations

While TFLOW offers an efficient alternative to text-based multi-agent communication, it also has several limitations.

First, the communication channel is less interpretable than natural language. In text-based MAS, intermediate messages can be inspected by humans to understand what each agent contributed. In contrast, TFLOW transmits information through low-rank weight perturbations, whose semantic content is difficult to directly interpret. This may complicate debugging, attribution, and safety auditing, especially when the generated perturbations lead to unexpected receiver behavior.

Second, the performance gap on HumanEval+ indicates that TFLOW cannot fully recover the benefits that text-based MAS may obtain from increased generation length. Text-based collaboration not only transmits inter-agent information but also expands the overall reasoning and refinement budget through explicit intermediate rationales, plans, and deliberation traces. These additional tokens can directly benefit tasks such as code synthesis, where longer solutions, iterative correction, and detailed implementation planning are often important. TFLOW avoids passing sender-written messages to the receiver and therefore removes much of this extra generation overhead. This improves efficiency, but it also means that TFLOW may not capture all gains arising from the longer generation process used by TextMAS. Future work may explore hybrid systems that combine concise textual messages with transient parameter perturbations to balance interpretability, generation budget, and efficiency.

B Additional Method Details

This appendix provides additional details for TFLOW that are omitted from the main text for brevity. We describe sender conditioning and layer aggregation, the parameter-generator architecture, sender fusion and transient receiver injection, the training objective, optimization algorithms, and computation cost.

B.1 Notation

We use N to denote the number of agents. Agents $A_1, \dots, A_{N-1}$ are senders and A_N is the receiver. All agents share the same frozen backbone parameters θ and are distinguished by role-specific prompts p_i . For a query q , sender A_i processes the prompted input (p_i, q) and produces hidden states $\{\mathbf{H}_i^{(l)}\}_{l=0}^L$, where

$$\mathbf{H}_i^{(l)} \in \mathbb{R}^{T_i \times d}.$$

Here T_i is the sender sequence length and d is the hidden dimension of the frozen backbone.

We use L for the number of targeted receiver layers and M for the number of targeted linear modules per targeted layer. The frozen weight of module m in layer l is denoted by

$$\mathbf{W}_m^{(l)} \in \mathbb{R}^{d_{\text{out}}^{(l,m)} \times d_{\text{in}}^{(l,m)}}.$$

The LoRA rank is r , and α is the LoRA scaling coefficient. The parameter generator has hidden dimension d_{pg} and N_{pg} transformer blocks.

B.2 Sender Conditioning and Layer Aggregation

For each query, every sender runs one frozen forward pass on its prompted input. We retain hidden states from all backbone layers:

$$\{\mathbf{H}_i^{(l)}\}_{l=0}^L, \quad \mathbf{H}_i^{(l)} \in \mathbb{R}^{T_i \times d}.$$

Since different layers encode complementary levels of abstraction, we aggregate them using learnable layer weights. Let λ_l be a scalar assigned to layer l , and let $\tau > 0$ be a temperature. The normalized layer weight is

$$\rho_l = \frac{\exp(\lambda_l/\tau)}{\sum_{l'=0}^L \exp(\lambda_{l'}/\tau)}. \quad (4)$$

The sender conditioning signal is then

$$\mathbf{C}_i = \sum_{l=0}^L \rho_l \mathbf{H}_i^{(l)} \in \mathbb{R}^{T_i \times d}. \quad (5)$$

This construction lets the generator access information from both shallow and deep sender representations while preserving a fixed conditioning dimension.

Before entering the generator, the conditioning signal is projected to the generator hidden dimension:

$$\tilde{\mathbf{C}}_i = \text{Proj}_{\text{in}}(\mathbf{C}_i) \in \mathbb{R}^{T_i \times d_{\text{pg}}}. \quad (6)$$

The sender backbone remains frozen, and gradients do not update θ .

B.3 Parameter Generator

The parameter generator $\mathcal{H}_p$ maps each sender conditioning signal $\mathbf{C}_i$ to LoRA factors for every targeted receiver module. For each layer $l \in \{1, \dots, L\}$ and module $m \in \{1, \dots, M\}$, it outputs

$$\mathbf{A}_{i,m}^{(l)} \in \mathbb{R}^{r \times d_{\text{in}}^{(l,m)}}, \quad \mathbf{B}_{i,m}^{(l)} \in \mathbb{R}^{d_{\text{out}}^{(l,m)} \times r}. \quad (7)$$

Thus, the corresponding low-rank update has shape

$$\mathbf{B}_{i,m}^{(l)} \mathbf{A}_{i,m}^{(l)} \in \mathbb{R}^{d_{\text{out}}^{(l,m)} \times d_{\text{in}}^{(l,m)}}.$$

Token-grid initialization. The generator starts from a learnable token grid

$$\mathbf{Q}_{\text{grid}} \in \mathbb{R}^{L \times HW \times d_{\text{pg}}},$$

where $H = H_A + H_B$ separates tokens used for generating **A**- and **B**-factors, and W indexes module–rank slots. The projected sender condition $\tilde{\mathbf{C}}_i$ initializes this grid through cross-attention:

$$\mathbf{Z}_i^{(0)} = \text{CA}(\mathbf{Q}_{\text{grid}}, \tilde{\mathbf{C}}_i, \tilde{\mathbf{C}}_i) + \mathbf{T}_{\text{layer}} + \mathbf{T}_{\text{slot}}, \quad (8)$$

where $\mathbf{T}_{\text{layer}}$ encodes the target receiver-layer index and $\mathbf{T}_{\text{slot}}$ encodes module and rank-slot positions.

Multi-axis transformer trunk. The initialized grid is refined by N_{pg} transformer blocks. Each block contains cross-layer self-attention, intra-layer self-attention, conditioning cross-attention, and a feed-forward network. For block t , we write

$$\mathbf{U}_i^{(t)} = \mathbf{Z}_i^{(t)} + \text{SA}_L(\text{LN}(\mathbf{Z}_i^{(t)})), \quad (9)$$

$$\mathbf{V}_i^{(t)} = \mathbf{U}_i^{(t)} + \text{SA}_{HW}(\text{LN}(\mathbf{U}_i^{(t)})), \quad (10)$$

$$\mathbf{Y}_i^{(t)} = \mathbf{V}_i^{(t)} + \text{CA}(\text{LN}(\mathbf{V}_i^{(t)}), \tilde{\mathbf{C}}_i, \tilde{\mathbf{C}}_i), \quad (11)$$

$$\mathbf{Z}_i^{(t+1)} = \mathbf{Y}_i^{(t)} + \text{FFN}(\text{LN}(\mathbf{Y}_i^{(t)})). \quad (12)$$

Here, SA_L attends across targeted receiver layers while batching over the HW axis, and SA_{HW} attends within each layer across module–rank slots. The conditioning cross-attention re-grounds the parameter tokens in the sender representations at every block. RoPE is applied along the active attention axis.

Detokenization into LoRA factors. After the final generator block, the grid is split along the H axis into **A**-tokens and **B**-tokens:

$$\mathbf{Z}_i^{(N_{\text{pg}})} = [\mathbf{Z}_{i,A}, \mathbf{Z}_{i,B}].$$

Two linear heads project these tokens into parameter slices:

$$\mathbf{T}_{i,A} = h_A(\mathbf{Z}_{i,A}), \quad \mathbf{T}_{i,B} = h_B(\mathbf{Z}_{i,B}). \quad (13)$$

The slices are rearranged according to layer, module, and rank-slot indices to produce

$$\left\{ \mathbf{A}_{i,m}^{(l)}, \mathbf{B}_{i,m}^{(l)} \right\}_{l=1, m=1}^{L, M}.$$

All generator parameters are shared across senders. Sender-specific LoRA factors arise solely from sender-specific conditioning signals $\mathbf{C}_i$.

B.4 Sender Fusion and Transient Injection

When multiple senders are active, TFlow fuses their generated updates at the perturbation level. This is important because fusing $\mathbf{A}$ - and $\mathbf{B}$ -factors separately would create unintended cross-sender products. Specifically,

$$\left(\sum_i \gamma_i \mathbf{B}_i\right) \left(\sum_j \gamma_j \mathbf{A}_j\right) = \sum_{i,j} \gamma_i \gamma_j \mathbf{B}_i \mathbf{A}_j,$$

which includes terms $\mathbf{B}_i \mathbf{A}_j$ for $i \neq j$. We therefore fuse complete low-rank perturbations $\mathbf{B}_i \mathbf{A}_i$.

The sender-fusion gate predicts one scalar score for each sender:

$$s_i = g_{\mathbf{w}}(\mathbf{C}_i). \quad (14)$$

Scores are normalized across active senders:

$$\gamma_i = \frac{\exp(s_i)}{\sum_{j=1}^{N-1} \exp(s_j)}. \quad (15)$$

The fused transient update for module m in layer l is

$$\Delta \mathbf{W}_m^{(l)} = \frac{\alpha}{r} \sum_{i=1}^{N-1} \gamma_i \mathbf{B}_{i,m}^{(l)} \mathbf{A}_{i,m}^{(l)}. \quad (16)$$

For a receiver activation matrix $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{in}}^{(l,m)}}$, the patched linear map is

$$\mathbf{X} \mapsto \mathbf{X}(\mathbf{W}_m^{(l)})^\top + \mathbf{X}(\Delta \mathbf{W}_m^{(l)})^\top. \quad (17)$$

The dense matrix $\Delta \mathbf{W}_m^{(l)}$ need not be explicitly materialized. The additive branch can be computed in low-rank form:

$$\mathbf{X}(\Delta \mathbf{W}_m^{(l)})^\top = \frac{\alpha}{r} \sum_{i=1}^{N-1} \gamma_i (\mathbf{X}(\mathbf{A}_{i,m}^{(l)})^\top) (\mathbf{B}_{i,m}^{(l)})^\top. \quad (18)$$

The frozen weight tensor $\mathbf{W}_m^{(l)}$ is never overwritten. The update exists only during the current receiver forward pass and is removed immediately afterward.

B.5 Training Objective

The trainable parameter set is

$$\Theta = \{\psi, \text{Proj}_{\text{in}}, \{\lambda_l\}_{l=0}^{L_{\text{total}}}, \mathbf{w}\},$$

where ψ denotes the parameter-generator weights, Proj_{in} denotes the conditioning projection, $\{\lambda_l\}$ are the layer-aggregation scalars, and $\mathbf{w}$ denotes the fusion-gate parameters. The frozen backbone parameters θ are excluded from Θ .

Task loss. For a training tuple (x_q, t, σ) , where x_q is the query-side input, t is the target completion, and σ is the data-source identifier, the task loss is the masked next-token cross-entropy over completion tokens:

$$\mathcal{L}_{\text{task}}(\Theta; x_q, t) = - \sum_{u \in \mathcal{I}_{\text{ans}}} \log p_{\theta, \Theta}(t_u \mid t_{<u}, x_q), \quad (19)$$

where $\mathcal{I}_{\text{ans}}$ denotes answer-token positions. The distribution $p_{\theta, \Theta}$ is computed by the frozen receiver equipped with the transient updates produced by Θ .

Flattened generated update. To define the diversity regularizer, we flatten all fused updates for the current input:

$$v_{\Theta}(x_q) = \text{vec} \left(\left\{ \Delta \mathbf{W}_m^{(l)}(x_q) \right\}_{l=1, m=1}^{L, M} \right). \quad (20)$$

In implementation, this vector can be represented either by explicitly flattened dense offsets or by an equivalent flattened low-rank representation, as long as the same representation is used consistently for the current vector and the cache.

Cache-based diversity regularizer. For each data source σ , we maintain a stop-gradient cache vector $v_{\text{prev}}^{(\sigma)}$, corresponding to the generated update from the previous optimization step involving that source. The diversity regularizer is

$$\mathcal{L}_{\text{div}}(\Theta; x_q) = \cos^2(v_{\Theta}(x_q), v_{\text{prev}}^{(\sigma(x_q))}). \quad (21)$$

This objective penalizes both parallel and anti-parallel collapse and is minimized when the current update is orthogonal to the cached update. Since $v_{\text{prev}}^{(\sigma)}$ is detached, the regularizer does not introduce an additional forward pass.

The total objective is

$$\mathcal{L}(\Theta) = \mathbb{E}_{(x_q, t, \sigma) \sim \mathcal{D}} [\mathcal{L}_{\text{task}}(\Theta; x_q, t) + \lambda_{\text{div}} \mathcal{L}_{\text{div}}(\Theta; x_q)]. \quad (22)$$

B.6 Training Algorithm

Algorithm 1 Training TFLOW

Require: Frozen backbone f_{θ} ; trainable parameters Θ ; sender prompts $\{p_i\}_{i=1}^{N-1}$; receiver prompt p_N ; dataset $\mathcal{D}$; diversity weight λ_{div} .

- 1: Initialize diversity cache $\{v_{\text{prev}}^{(\sigma)}\}$ for each data source σ .
 - 2: **for** each minibatch $\mathcal{B} \subset \mathcal{D}$ **do**
 - 3: Initialize minibatch loss $\mathcal{L}_{\mathcal{B}} \leftarrow 0$.
 - 4: **for** each example $(x_q, t, \sigma) \in \mathcal{B}$ **do**
 - 5: **for** each sender $i = 1, \dots, N - 1$ **do**
 - 6: Run frozen sender forward on (p_i, x_q) and collect $\{\mathbf{H}_i^{(l)}\}_{l=0}^{L_{\text{total}}}$.
 - 7: Aggregate hidden states into $\mathbf{C}_i$ using Equation (5).
 - 8: Generate LoRA factors $\{\mathbf{A}_{i,m}^{(l)}, \mathbf{B}_{i,m}^{(l)}\}_{l,m} \leftarrow \mathcal{H}_{\psi}(\mathbf{C}_i)$.
 - 9: **end for**
 - 10: Compute gate weights $\gamma_i = \text{softmax}_i(g_{\mathbf{w}}(\mathbf{C}_i))$.
 - 11: Fuse sender updates into $\Delta \mathbf{W}_m^{(l)}$ for all targeted modules using Equation (16).
 - 12: Temporarily patch the targeted receiver modules.
 - 13: Run the frozen receiver on (p_N, x_q, t) and compute $\mathcal{L}_{\text{task}}$ over completion tokens.
 - 14: Form $v_{\Theta}(x_q)$ and compute $\mathcal{L}_{\text{div}}$ using the stop-gradient cache $v_{\text{prev}}^{(\sigma)}$.
 - 15: Accumulate $\mathcal{L}_{\mathcal{B}} \leftarrow \mathcal{L}_{\mathcal{B}} + \mathcal{L}_{\text{task}} + \lambda_{\text{div}} \mathcal{L}_{\text{div}}$.
 - 16: Remove the temporary receiver patches.
 - 17: **end for**
 - 18: Update Θ using gradients of $\mathcal{L}_{\mathcal{B}}$.
 - 19: Update each used cache entry $v_{\text{prev}}^{(\sigma)} \leftarrow \text{stopgrad}(v_{\Theta}(x_q))$.
 - 20: **end for**
-

B.7 Inference Algorithm

At inference time, TFLOW follows the same sender-conditioning and parameter-generation process, but omits the task loss, diversity regularizer, and cache update. The generated perturbation is used only during receiver decoding.

B.8 Computation Cost

The additional cost of TFLOW beyond a standard receiver forward pass comes from sender forwarding, parameter generation, and transient LoRA injection.

Sender forward passes. Let $C_{\text{bb}}(T)$ denote the cost of one frozen backbone forward pass with sequence length T . The $N - 1$ senders incur

$$C_{\text{sender}} = \sum_{i=1}^{N-1} C_{\text{bb}}(T_i). \quad (23)$$

These sender passes are independent and can be parallelized. They also do not require autoregressive decoding; each sender only produces hidden states for conditioning.

Algorithm 2 Inference with TFlow

Require: Frozen backbone f_θ ; trained parameters Θ ; sender prompts $\{p_i\}_{i=1}^{N-1}$; receiver prompt p_N ; query q .

- 1: **for** each sender $i = 1, \dots, N - 1$ **do**
 - 2: Run frozen sender forward on (p_i, q) and collect $\{\mathbf{H}_i^{(l)}\}_{l=0}^{L_{\text{total}}}$.
 - 3: Aggregate hidden states into $\mathbf{C}_i$.
 - 4: Generate LoRA factors $\{\mathbf{A}_{i,m}^{(l)}, \mathbf{B}_{i,m}^{(l)}\}_{l,m} \leftarrow \mathcal{H}_\psi(\mathbf{C}_i)$.
 - 5: **end for**
 - 6: Compute $\gamma_i = \text{softmax}_i(g_{\mathbf{w}}(\mathbf{C}_i))$.
 - 7: Fuse updates into $\Delta \mathbf{W}_m^{(l)} = \frac{\alpha}{r} \sum_{i=1}^{N-1} \gamma_i \mathbf{B}_{i,m}^{(l)} \mathbf{A}_{i,m}^{(l)}$.
 - 8: Temporarily patch the targeted receiver modules.
 - 9: Decode the final answer using receiver A_N conditioned on (p_N, q) .
 - 10: Remove all temporary patches.
-

Parameter generation. Let $S = HW$ be the number of parameter tokens per targeted layer. Each generator block applies cross-layer attention, intra-layer attention, conditioning cross-attention, and an FFN. For sender i , the cross-layer attention costs $O(SL^2d_{\text{pg}})$, the intra-layer attention costs $O(LS^2d_{\text{pg}})$, the conditioning cross-attention costs $O(LST_id_{\text{pg}})$, and the FFN costs $O(LSd_{\text{pg}}^2)$. Therefore,

$$C_{\text{gen}}^{(i)} = O(N_{\text{pg}} [SL^2d_{\text{pg}} + LS^2d_{\text{pg}} + LST_id_{\text{pg}} + LSd_{\text{pg}}^2]), \quad (24)$$

and across all senders,

$$C_{\text{gen}} = \sum_{i=1}^{N-1} C_{\text{gen}}^{(i)}. \quad (25)$$

Transient LoRA injection. For a targeted linear module with receiver sequence length T , input dimension $d_{\text{in}}^{(l,m)}$, output dimension $d_{\text{out}}^{(l,m)}$, and LoRA rank r , the low-rank branch in Equation (18) costs

$$O((N-1)Tr(d_{\text{in}}^{(l,m)} + d_{\text{out}}^{(l,m)})). \quad (26)$$

This is small compared with the dense projection cost

$$O(Td_{\text{in}}^{(l,m)}d_{\text{out}}^{(l,m)})$$

when $r \ll \min(d_{\text{in}}^{(l,m)}, d_{\text{out}}^{(l,m)})$. Summing over all targeted modules gives

$$C_{\text{inj}} = O\left((N-1)Tr \sum_{l=1}^L \sum_{m=1}^M (d_{\text{in}}^{(l,m)} + d_{\text{out}}^{(l,m)})\right). \quad (27)$$

Overall overhead. The total additional cost is

$$C_{\text{extra}} = C_{\text{sender}} + C_{\text{gen}} + C_{\text{inj}}. \quad (28)$$

The sender and generator costs scale linearly with the number of active senders $N - 1$, and the injection cost scales linearly with both $N - 1$ and the LoRA rank r . In practice, sender forward passes and per-sender generation are independent and can be batched or parallelized, reducing wall-clock overhead.